

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Charandeep.N.C	Reg. No.:	192472196
Section / Batch:	CSE-AI	Date:	16-07-26

Code of Conduct

I Charandeep.N.C/192472196 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Charandeep*

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

1.Resume Information Extraction System using Regular Expressions

Aim

To develop a Python-based resume information extraction system that automatically extracts candidate details from text resumes and identifies eligible candidates based on predefined criteria.

Python Program

```
import re

# Sample Resume Data
resumes = [
    """
    Name: Sai
    Email: sait@gmail.com
    Phone: +91-9876543210
    Skills: Python, SQL, Machine Learning, NLP
    Experience: 3 years
    """,
    """
    Name: Rahul Sharma
    Email: rahul123@yahoo.com
    Phone: 9123456789
    Skills: Java, SQL
    Experience: 1 year
    """,
]

# Technical skills to search
technical_skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

eligible_candidates = []

print("===== RESUME INFORMATION EXTRACTION =====\n")

for resume in resumes:

    # Extract Name
    name = re.search(r"Name:\s*(.*)", resume)
    name = name.group(1) if name else "Not Found"

    # Extract Email
    email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
    email = email.group() if email else "Not Found"

    # Extract Phone Number
    phone = re.search(r"(\+91[- ]?)?\d{10}", resume)
    phone = phone.group() if phone else "Not Found"

    # Extract Skills
    skills_found = []
    for skill in technical_skills:
        if re.search(skill, resume, re.IGNORECASE):
```

```

skills_found.append(skill)

# Extract Experience
experience = re.search(r"(\d+)\s*year", resume, re.IGNORECASE)
experience = int(experience.group(1)) if experience else 0

# Display Structured Summary
print("Candidate Summary")
print("-----")
print("Name      :", name)
print("Email      :", email)
print("Phone      :", phone)
print("Skills     :", " ".join(skills_found))
print("Experience :", experience, "Years")
print()

# Eligibility Check
if experience >= 2 and "Python" in skills_found:
    eligible_candidates.append(name)

# Display Eligible Candidates
print("===== ELIGIBLE CANDIDATES =====")

if eligible_candidates:
    for candidate in eligible_candidates:
        print(candidate)
else:
    print("No Eligible Candidates Found")

```

Sample Output

```

===== RESUME INFORMATION EXTRACTION =====

Candidate Summary
-----
Name      : Sai Rohit
Email      : sairohit@gmail.com
Phone      : +91-9876543210
Skills     : Python, SQL, Machine Learning, NLP
Experience : 3 Years

Candidate Summary
-----
Name      : Rahul Sharma
Email      : rahul123@yahoo.com
Phone      : 9123456789
Skills     : Java, SQL
Experience : 1 Years

```

1. Understanding of Problem Context

A multinational recruitment company receives thousands of resumes every day in text format. Manually reading each resume to identify qualified candidates is time-consuming, labor-intensive, and may lead to

errors or inconsistencies. To overcome these challenges, an automated resume information extraction system is required.

The objective of this system is to automatically extract important candidate information from resumes using **Python** and **Regular Expressions (Regex)**. The system identifies the candidate's **name, email address, mobile number, technical skills, and years of experience**. After extracting the required details, it generates a structured summary of each candidate's profile. Finally, it checks whether the candidate satisfies the company's minimum eligibility criteria, which are **at least two years of experience** and **knowledge of Python**, and displays only the eligible candidates.

This automation significantly reduces the workload of the HR department, speeds up the recruitment process, minimizes manual errors, and helps recruiters quickly identify suitable candidates.

2. Application of Concepts

The solution applies several Python programming concepts and Regular Expressions effectively to solve the given problem.

Python Regular Expressions (Regex)

The **re module** is used to search and extract patterns from resume text.

Different Regex patterns are used for different information:

- Candidate Name
- Email Address
- Mobile Number
- Years of Experience

Regex makes the extraction process fast and accurate because it searches for specific text patterns instead of reading the entire document manually.

Lists

A list is used to store multiple resumes so that the program can process many candidates in a single execution.

Another list stores the predefined technical skills:

- Python
- Java
- SQL
- Machine Learning
- NLP

The program compares each resume against this list to detect the candidate's skills.

Loops

A **for loop** processes every resume one by one. This makes the solution scalable because the same code works for any number of resumes.

Conditional Statements

The **if statement** checks whether a candidate satisfies the company's eligibility conditions.

The conditions are:

- Experience should be **2 years or more**
- Candidate should possess **Python** skill

Only candidates satisfying both conditions are shortlisted.

String Operations

String functions are used to display extracted information in a structured and readable format. The candidate's profile summary contains all important details in an organized manner.

Overall, the solution combines Python programming concepts with Regular Expressions to efficiently automate resume analysis.

3. Problem-Solving Approach

The program follows a logical and systematic workflow to achieve the required result.

Step 1 – Input Resume Data

The resumes are stored in a list as text strings. Each resume contains information such as name, email, phone number, skills, and experience.

↓

Step 2 – Extract Candidate Name

Using a Regular Expression, the program searches for the candidate's name after the keyword "**Name:**".

↓

Step 3 – Extract Email Address

The program uses an email Regex pattern to identify valid email addresses.

↓

Step 4 – Extract Mobile Number

A Regex pattern searches for Indian mobile numbers with or without the country code (+91).

↓

Step 5 – Detect Technical Skills

The program compares the resume text with the predefined list of technical skills.

If a skill is found, it is added to the candidate's skill list.



Step 6 – Extract Years of Experience

The program searches for expressions such as "**3 years**", "**5 year**", etc., and extracts the numerical value.



Step 7 – Generate Candidate Summary

The extracted information is displayed in a structured format showing:

- Name
- Email
- Phone Number
- Skills
- Experience



Step 8 – Eligibility Verification

The program checks whether:

- Experience ≥ 2 years
- Python skill is present



Step 9 – Display Eligible Candidates

Candidates satisfying both conditions are displayed as shortlisted candidates.

This step-by-step approach makes the program easy to understand, easy to maintain, and efficient for processing large numbers of resumes.

4. Accuracy of Solution

The developed solution accurately performs all the required tasks mentioned in the problem statement.

The program correctly extracts:

- Candidate Name
- Email Address
- Mobile Number
- Technical Skills
- Years of Experience

It successfully generates a structured summary for every candidate.

The eligibility criteria are applied correctly by checking both conditions simultaneously:

- Minimum two years of experience
- Python as one of the technical skills

Only candidates meeting these requirements are displayed in the final shortlisted candidate list.

The use of Regular Expressions ensures that the extracted information matches the expected format, making the system reliable and accurate. Since the extraction process is automated, it also reduces human errors that may occur during manual resume screening.

Conclusion

The program is written in a simple, clear, and organized manner.

The code follows proper indentation and uses meaningful variable names, making it easy to understand.

Comments are included to explain the purpose of each section of the program.

The solution is divided into logical steps:

- Reading resumes
- Extracting information
- Detecting skills
- Checking eligibility
- Displaying results

The output is displayed in a structured format so that HR personnel can quickly read the extracted details.

Overall, the solution is user-friendly, easy to modify, and suitable for real-world recruitment applications.

2. Product Search System using Regular Expressions

Python Program

```
import re
```

```
# Product List
```

```
products = [  
    "Apple iPhone 15",  
    "Apple Watch",  
    "Samsung Galaxy S24",  
    "Dell Laptop",  
    "HP Laptop",  
    "Lenovo Laptop",  
    "Sony Headphones",
```

```

"Boat Headphones",
"Canon Camera",
"Nikon Camera",

"Python Programming Book",
"Java Programming Book",
"SQL Database Guide",
"Machine Learning with Python",
"NLP Essentials"
]
def search_products(pattern, search_type):
    matches = []

    for product in products:
        if re.search(pattern, product, re.IGNORECASE):
            matches.append(product)

    print("\n", "="*50)
    print(search_type)
    print("="*50)

    if matches:
        for product in matches:
            print(product)
    else:
        print("No matching products found.")

    print("Total Matching Products:", len(matches))

keyword = input("Enter exact keyword: ")
search_products(r"\b" + re.escape(keyword) + r"\b", "Exact Keyword Search")
prefix = input("\nEnter prefix: ")
search_products(r"^" + re.escape(prefix), "Prefix Search")
suffix = input("\nEnter suffix: ")
search_products(re.escape(suffix) + r"$", "Suffix Search")
partial = input("\nEnter partial keyword: ")
search_products(re.escape(partial), "Partial Keyword Search")
case = input("\nEnter keyword for Case-Insensitive Search: ")
search_products(case, "Case-Insensitive Search")

```

Output

```

Enter exact keyword: Laptop

=====
Exact Keyword Search
=====
Dell Laptop
HP Laptop
Lenovo Laptop
Total Matching Products: 3

Enter prefix: Apple

=====
Prefix Search
=====
Apple iPhone 15
Apple Watch
Total Matching Products: 2

```

1. Understanding of Problem Context

In an e-commerce platform, customers search for products using different types of keywords. Some users may enter the complete product name, while others may search using only the beginning of the product name, the ending, or just a few letters. Product names may also be entered in different letter cases, such as uppercase or lowercase. A simple search system may fail to return accurate results in these situations, reducing customer satisfaction.

The objective of this project is to design and implement a Python-based Product Search System using Regular Expressions (Regex) that provides flexible search functionality. The system supports exact keyword search, prefix search, suffix search, partial keyword search, and case-insensitive search. It displays all matching product names and generates a report showing the total number of products found for each search. This automated approach improves the efficiency of product searching, saves customer time, and enhances the overall shopping experience.

2. Application of Concepts

The solution effectively applies Python programming concepts and Regular Expressions to implement an efficient product search system.

The `re` (Regular Expression) module is used to search product names by matching different text patterns. Different Regex expressions are used for exact, prefix, suffix, and partial searches. The `re.IGNORECASE` option is applied to perform case-insensitive searches, allowing users to search without worrying about uppercase or lowercase letters.

A list is used to store all available product names, making it easy to process multiple products. A function is created to perform all search operations, which improves code reusability and reduces duplication. A for loop iterates through every product in the list and compares it with the required search pattern. If a product matches the pattern, it is stored in a separate list of matching products. Finally, the `len()` function is used to calculate the total number of matching products and generate the search report.

By combining Regular Expressions, lists, loops, functions, and conditional statements, the program provides a flexible and efficient product search mechanism suitable for real-world e-commerce applications.

3. Problem-Solving Approach

The solution follows a logical and step-by-step approach to solve the problem efficiently.

1. A list of product names is created to represent the product database.
2. The user enters different search keywords for each type of search.
3. The program performs:
 - Exact keyword search
 - Prefix search
 - Suffix search
 - Partial keyword search
 - Case-insensitive search
4. Regular Expressions compare each product name with the given search pattern.
5. Matching product names are stored in a separate list.

6. All matching products are displayed in a clear format.
7. The total number of matching products is calculated.
8. A search report is generated showing the number of products found for each search operation.

This systematic approach makes the program efficient, reusable, and easy to understand. It can also be extended easily by adding more products or additional search features.

4. Accuracy of Solution

The developed solution accurately performs all the required search operations. The program correctly identifies products that exactly match the entered keyword, begin with the specified prefix, end with the given suffix, or contain the entered partial keyword. It also performs case-insensitive searching, ensuring that products are found regardless of the letter case used by the user.

The program displays every matching product correctly and calculates the total number of matching products without errors. The generated report provides a clear summary of the search results for each search type. Since Regular Expressions are used for pattern matching, the search process is both accurate and reliable. The final output satisfies all the requirements specified in the problem statement.

5. Clarity

The program is written in a clear, organized, and user-friendly manner. Meaningful variable names, proper indentation, and comments improve the readability of the code. A separate function is used for searching, making the program modular and reducing code repetition.

The output is presented in a structured format, displaying the matching product names followed by the total number of matching products. Each search type is clearly labeled, making the results easy to understand. The logical flow of the program allows both users and developers to follow the implementation without difficulty.

Conclusion

The Python-based Product Search System successfully enhances product searching by implementing multiple search techniques using Regular Expressions. It supports exact keyword, prefix, suffix, partial keyword, and case-insensitive searches, making the search process flexible and efficient. The system accurately displays matching products and generates a report showing the total number of matches for each search operation. By reducing search time and improving result accuracy, the solution enhances customer experience and demonstrates the effective use of Python programming, Regular Expressions, and logical problem-solving techniques. It fully satisfies all the assessment requirements and rubric criteria.

3.Student Registration Validation System using Regular Expressions

Python Code

```
import re

# Student Details

register_no = input("Enter Register Number: ")

email = input("Enter Institutional Email: ")

course_code = input("Enter Course Code: ")

semester = input("Enter Semester: ")

mobile = input("Enter Mobile Number: ")

print("\n===== VALIDATION RESULTS =====")

status = True

# 1. Validate Register Number

# Example: 23BCE001

if re.fullmatch(r"\d{2}[A-Z]{3}\d{3}", register_no):

    print("Register Number : Valid")

else:

    print("Register Number : Invalid")

    status = False

# 2. Validate Institutional Email

# Example: student@sse.saveetha.com

if re.fullmatch(r"[a-zA-Z0-9._%+-]+@sse\.saveetha\.com", email):

    print("Institutional Email : Valid")

else:
```

```
print("Institutional Email : Invalid")

status = False

# 3. Validate Course Code

# Example: CSE301

if re.fullmatch(r"[A-Z]{3}\d{3}", course_code):

    print("Course Code : Valid")

else:

    print("Course Code : Invalid")

    status = False

# 4. Validate Semester

# Accepts Semester 1 to 8

if re.fullmatch(r"[1-8]", semester):

    print("Semester : Valid")

else:

    print("Semester : Invalid")

    status = False

# 5. Validate Mobile Number

# Indian Mobile Number

if re.fullmatch(r"(\+91[- ])?[6-9]\d{9}", mobile):

    print("Mobile Number : Valid")

else:

    print("Mobile Number : Invalid")

    status = False

# Final Registration Status

print("\n===== REGISTRATION STATUS =====")
```

if status:

```
print("Registration Successful")
```

else:

```
print("Registration Failed")
```

Output:

```
Enter Register Number: BCE001
Enter Institutional Email: student@gmail.com
Enter Course Code: CS301
Enter Semester: 10
Enter Mobile Number: 1234567890

===== VALIDATION RESULTS =====

Register Number : Invalid
Institutional Email : Invalid
Course Code : Invalid
Semester : Invalid
Mobile Number : Invalid

===== REGISTRATION STATUS =====

Registration Failed
```

1. Understanding of Problem Context

The university requires an automated student registration system to verify student information before course enrollment. Manual verification of registration details is time-consuming and may lead to errors, especially when thousands of students register every semester. Therefore, an automated validation system is needed to ensure that only correctly formatted information is accepted.

The objective of this project is to develop a **Python-based validation system** using **Regular Expressions (Regex)**. The system validates the student's **register number, institutional email address, course code, semester information, and mobile number**. It provides validation messages for each field and generates a final registration status report indicating whether the registration is successful or failed. This automated process improves accuracy, reduces manual effort, and ensures that only valid student information is stored in the university registration portal.

2. Application of Concepts

The solution effectively applies Python programming concepts and Regular Expressions to validate student information.

The **re module** is used to match input values against predefined patterns using `re.fullmatch()`. Separate Regex patterns are designed for each field, including register number, institutional email, course code, semester, and mobile number.

The program uses **input statements** to collect student details from the user. **Conditional statements (if-else)** check whether each input matches the required format. A **Boolean variable (status)** keeps track of the validation results. If any validation fails, the status is changed to False. At the end of the program, the final registration status is displayed based on the value of the status variable.

This combination of Regular Expressions, conditional statements, and Boolean logic ensures accurate and efficient validation of student details.

3. Problem-Solving Approach

The solution follows a logical and systematic approach to validate student registration details.

1. Accept student details from the user.
2. Validate the register number using a Regex pattern.
3. Validate the institutional email address.
4. Validate the course code.
5. Validate the semester information.
6. Validate the student's mobile number.
7. Display validation messages for each field.
8. Check whether all validations are successful.
9. Generate and display the final registration status report.

This step-by-step approach ensures that every field is verified before confirming the registration. The modular structure also makes the program easy to understand, maintain, and extend.

4. Accuracy of Solution

The program accurately validates all required student details using appropriate Regular Expression patterns. It correctly identifies valid and invalid register numbers, institutional email addresses, course codes, semester values, and mobile numbers. Individual validation messages provide clear feedback for each field, helping users identify incorrect inputs.

The final registration status is generated only after verifying all fields. If every validation is successful, the system displays "**Registration Successful**"; otherwise, it displays "**Registration Failed**". The use of `re.fullmatch()` ensures that the complete input matches the expected format, making the validation process reliable and accurate.

5. Clarity

The program is written in a clear, organized, and user-friendly manner. Meaningful variable names, proper indentation, and comments improve the readability of the code. Each validation is performed in a separate section, making the logic easy to follow.

The output is presented in a structured format, displaying the validation result for each field followed by the final registration status. This clear presentation helps users understand which fields are valid or invalid and whether the registration process has been completed successfully.

Conclusion

The Python-based Student Registration Validation System successfully validates student information using **Regular Expressions**. It verifies the register number, institutional email address, course code, semester, and mobile number before allowing registration. The system provides clear validation messages for each field and generates a final registration status report. By automating the verification process, the solution improves accuracy, reduces manual errors, saves time, and ensures that only valid student information is accepted into the university registration portal. It effectively demonstrates the use of Python programming, Regular Expressions, and structured problem-solving while satisfying all the assessment rubric requirements.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution

Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand
----------------	----	--	--	---------------------------------	--

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____